Generación acelerada en GPU de campos de distancia con signo (SDF) a partir de mallas poligonales

Evaluación de técnicas de propagación y aplicación a iluminación global interactiva



Universitat Oberta de Catalunya

Daniel Martí Casanova

Máster Universitario en Diseño y Programación de Videojuegos
Programación Gráfica y Motores de Videojuegos

Nombre Tutor/a de TF

Gonzalo Nicolás Barreales

Profesor/a responsable de la asignatura

Joan Arnedo Moreno

8/11/2026



Esta obra está sujeta a una licencia de Reconocimiento-NoComercial-SinObraDerivada 3.0 España de Creative Commons.

FICHA DEL TRABAJO FINAL

Título del trabajo:	Generación acelerada en GPU de campos de distancia con signo (SDF) a partir de mallas poligonales
Nombre del autor:	Daniel Martí Casanova
Nombre del tutor/a de TF:	Gonzalo Nicolás Barreales
Nombre del PRA:	Joan Arnedo Moreno
Fecha de entrega (mm/aaaa):	8/11/2026
Titulación o programa:	Máster Universitario en Diseño y Programación de Videojuegos
Área del Trabajo Final:	Programación Gráfica y Motores de Videojuegos
Idioma del trabajo:	Castellano
Palabras clave	Campos de distancia con signo, generación acelerada en GPU, iluminación global

Resumen del Trabajo

Este Trabajo Final de Máster propone diseñar, implementar y evaluar dos métodos de generación de campos de distancia con signo (*Signed Distance Fields*, SDF) volumétricos en GPU a partir de mallas poligonales. Estas representaciones permiten realizar consultas de distancia para el trazado de rayos por software, pero su generación puede resultar costosa al aumentar la resolución del volumen y la complejidad de la geometría.

Se desarrollará un método base de cálculo de distancias mediante *compute shaders* y una alternativa que combine la captura de datos de superficie por rasterización con el algoritmo *Jump Flooding* (JFA) en tres dimensiones. El método base se comprobará con geometrías de control antes de utilizarlo como referencia.

Los campos generados se integrarán en un visor interactivo construido a partir de un prototipo previo del autor. El visor permitirá examinar el volumen y observar su aplicación a iluminación indirecta difusa de un rebote, oclusión ambiental y sombras suaves. Su función será apoyar la evaluación, sin constituir un motor gráfico de propósito general.

La comparación analizará el tiempo de generación, el consumo de memoria de vídeo y el error de distancia sobre un conjunto delimitado de escenas. Se espera identificar las ventajas y limitaciones de cada método, sin presuponer una mejora uniforme en todas las condiciones. Esta segunda entrega consolida la introducción y presenta el estado del arte, la estrategia de desarrollo, las herramientas, el protocolo experimental y el visor interactivo de validación ya implementado.

Abstract

This master's thesis proposes the design, implementation, and evaluation of two GPU methods for generating volumetric Signed Distance Fields (SDFs) from polygonal meshes. These representations support distance queries for software ray tracing, but their generation can become computationally expensive as volume resolution and geometric complexity increase.

A baseline distance computation method will be implemented using compute shaders. An alternative approach will combine rasterized surface data with the three-dimensional Jump Flooding Algorithm (JFA). The baseline will first be checked against control geometries with known distances before being used as a reference.

The generated fields will be integrated into an interactive viewer adapted from a prototype previously developed by the author. The viewer will support volume inspection and demonstrate one-bounce diffuse indirect lighting, ambient occlusion, and soft shadows. It will serve as an evaluation tool rather than a general-purpose graphics engine.

The experimental comparison will examine generation time, video memory consumption, and distance error across a limited set of test scenes. The aim is to identify the advantages and limitations of each method without assuming consistent improvements under all conditions. This second submission presents the state of the art, the development strategy, the experimental protocol, and the interactive viewer for validation.

Repositorio del código / Code repository: https://github.com/damacaa/mesh-to-sdf-gpu

Declaración de uso de recursos de terceros

Bibliotecas y recursos de software de terceros

Para el desarrollo de la aplicación y de los procesos de renderizado y cálculo en GPU se han utilizado C++20 y OpenGL 4.6 junto con las siguientes bibliotecas y recursos de terceros:

- **SDL3:** creación de ventanas, gestión del contexto gráfico OpenGL y tratamiento de eventos de entrada.
- **GLAD:** carga dinámica de punteros a funciones del perfil Core de OpenGL 4.6.
- **GLM:** operaciones de álgebra lineal, matrices y transformaciones espaciales.
- **Assimp:** importación y preprocesamiento de mallas y escenas tridimensionales.
- **Dear ImGui:** interfaz gráfica de usuario para modificar parámetros en tiempo real, seleccionar modos de visualización y consultar métricas.
- **stb_image / stb_image_write:** carga de texturas auxiliares y exportación de capturas de pantalla sin pérdida.
- **hg_sdf (Mercury):** funciones auxiliares de primitivas analíticas y operadores de geometría constructiva en GLSL empleadas en las escenas de control del visor.
- **NVIDIA Spatiotemporal Blue Noise (STBN):** máscaras precalculadas de ruido azul espaciotemporal utilizadas en el muestreo estocástico del *path tracer*.

Modelos y escenas tridimensionales

Se han utilizado **Cornell Box**, como escena controlada para examinar la iluminación indirecta y las sombras, y **Crytek Sponza**, para realizar las pruebas con mayor complejidad geométrica y oclusión. Estas escenas se han complementado con modelos geométricos sintéticos de control (esfera y cubo teselados a distintas densidades).

La selección de las versiones concretas y las adaptaciones realizadas se han ajustado a las condiciones de entrada admitidas por los generadores, documentando en la memoria la procedencia y las características de los modelos empleados.

Prototipo previo del autor

En este proyecto se ha reutilizado como base un prototipo propio de *path tracer* interactivo basado en *ray marching* sobre primitivas SDF analíticas definidas directamente en el shader.

Este desarrollo previo ha servido como punto de partida del visor interactivo y su adaptación ha permitido disponer de un entorno de pruebas validado para inspeccionar los campos volumétricos generados a partir de mallas.

La arquitectura de texturas tridimensionales en GPU, la generación de SDF a partir de mallas poligonales, la implementación de la alternativa basada en JFA en tres dimensiones, el *pipeline* de reducción de ruido espacio-temporal SVGF y la evaluación comparativa se han desarrollado específicamente durante el presente TFM, distinguiéndose de las funcionalidades procedentes del prototipo anterior.

Uso de herramientas de inteligencia artificial

Se declara el uso de Antigravity (modelos Gemini y Claude) y OpenAI (ChatGPT / Codex) como herramientas de apoyo para estructurar y revisar la redacción técnica de la memoria, asistir en la implementación y depuración de shaders en GLSL y código C++, y analizar alternativas algorítmicas. Entre los *prompts* e instrucciones empleados se incluyen consultas para la revisión estilística e impersonal del documento, el diagnóstico de artefactos numéricos en el filtro SVGF y la estructuración de pruebas automatizadas, cuyo registro detallado de sesiones se recoge en el archivo `docs/internal/declaracion_recursos_ia.md` del repositorio.

La revisión del contenido, la selección de los métodos y la validación de las implementaciones y de los resultados han sido responsabilidad exclusiva del autor. Las respuestas de estas herramientas no han sustituido la consulta de las fuentes bibliográficas originales ni la comprobación experimental.

Índice

1. Introducción 1
1.1. Contexto y justificación del trabajo 1
1.2. Objetivos del trabajo 3
1.2.1. Objetivo general 3
1.2.2. Objetivos específicos 3
1.2.3. Delimitación del alcance 4
1.3. Impacto en sostenibilidad, ético-social y de diversidad 4
1.3.1. Impacto en sostenibilidad 4
1.3.2. Impacto ético-social 5
1.3.3. Impacto en diversidad 5
1.4. Enfoque y método seguido 5
1.4.1. Marco de investigación 5
1.4.2. Metodología de producción del software 5
1.4.3. Gestión del proyecto 6
1.4.4. Distribución del trabajo entre entregas 6
1.5. Planificación del trabajo 6
1.5.1. Recursos necesarios 6
1.5.2. Dedicación y seguimiento 7
1.5.3. Hitos por PEC 7
1.5.4. Cronograma 8
1.5.5. Riesgos y medidas de mitigación 9
1.6. Breve sumario de productos obtenidos 9
1.7. Breve descripción de los otros capítulos de la memoria 10
1.7.1. Capítulo 2. Estado del arte 10
1.7.2. Capítulo 3. Materiales y métodos 10
1.7.3. Capítulo 4. Diseño 10
1.7.4. Capítulo 5. Resultados 10
1.7.5. Capítulo 6. Conclusiones y trabajo futuro 10
1.7.6. Capítulo 7. Glosario 11
1.7.7. Capítulo 8. Bibliografía 11
1.7.8. Capítulo 9. Anexos 11
2. Estado del arte 12
2.1. Campos de distancia con signo (SDF) y *sphere tracing* 12
2.2. Generación de SDF volumétricos en GPU a partir de mallas poligonales . . 12
2.2.1. Formulación matemática de la distancia mínima punto-triángulo en $\mathbb{R}^3$ 13
2.2.2. Paralelización masiva en GPU mediante Compute Shaders 14
2.2.3. Determinación del signo: mallas cerradas frente a mallas arbitrarias 14
2.3. Propagación espacial mediante Jump Flooding Algorithm (JFA) en 3D . . . 16
2.3.1. Estructura y fases del *pipeline* JFA en tres dimensiones 16
2.3.2. Análisis de errores de aproximación y variantes (JFA $+1$, $1+$ JFA) . 17
2.4. Iluminación global en tiempo real y reconstrucción espacio-temporal 17
2.5. Síntesis, aportación e hipótesis del trabajo 18

3. Materiales y métodos 20
3.1. Estrategia de desarrollo, alternativas y decisiones técnicas 20
3.2. Herramientas y pila tecnológica 21
3.3. Entorno interactivo de validación desarrollado (Hito OE1 y avance OE2) . . 21
3.4. Escenas de evaluación y protocolo experimental 24
3.4.1. Modelos y escenas poligonales de prueba 24
3.4.2. Protocolo de medición y métricas cuantitativas 25
4. Diseño 27
5. Resultados 28
6. Conclusiones y Trabajo futuro 29
6.1. Conclusiones 29
6.2. Seguimiento de la planificación 29
6.3. Líneas de futuro 29
7. Glosario 30
8. Bibliografía 32
9. Anexos 34

Lista de figuras

1. Ejemplo de iluminación global con transferencia de color (*color bleeding*): la luz rebota en el suelo e ilumina la parte inferior de la esfera; la luz reflejada en las paredes modula el sombreado de la esfera con el color de cada pared. 2
2. Arquitectura del banco de pruebas visual con evaluación de SDF y reducción de ruido SVGF. 22
3. Escena de validación Cornell Box renderizada en tiempo real en el visor a 1 muestra por píxel con reconstrucción SVGF. 24
4. Escena de validación con muestrario de primitivas SDF, reflexiones especulares y sombras suaves. 25

1. Introducción

1.1. Contexto y justificación del trabajo

Un campo de distancia con signo (*Signed Distance Field*, SDF) es una representación espacial que asigna a cada punto del espacio la distancia mínima hasta la superficie geométrica más cercana, utilizando el signo para distinguir el interior del exterior cuando la geometría admite esa clasificación. Esta formulación permite consultar de manera directa tanto la proximidad a una frontera como la dirección hacia ella a través del gradiente del campo. Gracias a estas propiedades, los SDF se utilizan en ámbitos diversos de la computación espacial más allá de la síntesis de imagen: en simulación física y detección de colisiones, donde los valores negativos cuantifican la profundidad de penetración entre cuerpos y el gradiente define la normal de contacto [1, 2]; y en robótica y visión por computador, donde permiten reconstruir el entorno a partir de sensores de profundidad para planificar trayectorias libres de obstáculos o simular la interacción física con objetos escaneados [3].

Dentro del desarrollo de videojuegos y la computación gráfica interactiva, además de su utilidad en motores de física, los SDF destacan como estructura de aceleración para el trazado de rayos por software mediante el algoritmo de *sphere tracing* [4], que adapta la longitud de los pasos de un rayo según la distancia segura a la superficie. Una de las aplicaciones más exigentes de este enfoque es la iluminación global en tiempo real, que plantea un compromiso constante entre calidad visual y coste de cálculo. Además de la luz que llega directamente desde una fuente hasta una superficie, la iluminación global considera las contribuciones indirectas producidas por las reflexiones sobre otras superficies de la escena, descritas por la ecuación de renderizado [5]. Estas interacciones permiten representar fenómenos como la transferencia de color entre objetos próximos, conocida como *color bleeding* (Figura 1).

En este contexto, sistemas de iluminación modernos como Lumen, de Unreal Engine 5 [6], constituyen una referencia del empleo de campos de distancia discretizados para realizar consultas espaciales sin depender exclusivamente de hardware dedicado de trazado de rayos. No obstante, en una representación volumétrica discretizada, la precisión del campo y los errores de aproximación pueden afectar a las consultas espaciales y producir artefactos visuales, por lo que su comportamiento debe evaluarse. El presente trabajo se centrará en la generación de dichos campos en GPU y en una demostración acotada de su uso en iluminación global, sin intentar reproducir un sistema de iluminación de la complejidad de Lumen.

La generación del volumen de distancias es uno de los problemas centrales que plantea el uso de SDF en todos estos dominios. Cuando se parte de mallas poligonales, el cálculo exacto de distancias entre los puntos de una cuadrícula tridimensional y los triángulos de una malla puede requerir un número elevado de evaluaciones, cuyo coste crece con la resolución del volumen y la complejidad de la geometría. Aunque el precálculo resulta suficiente para entornos estáticos, reducir el tiempo de generación permitiría explorar actualizaciones más frecuentes y ampliar sus condiciones de uso.

La motivación práctica del proyecto parte de un límite concreto de un prototipo previo

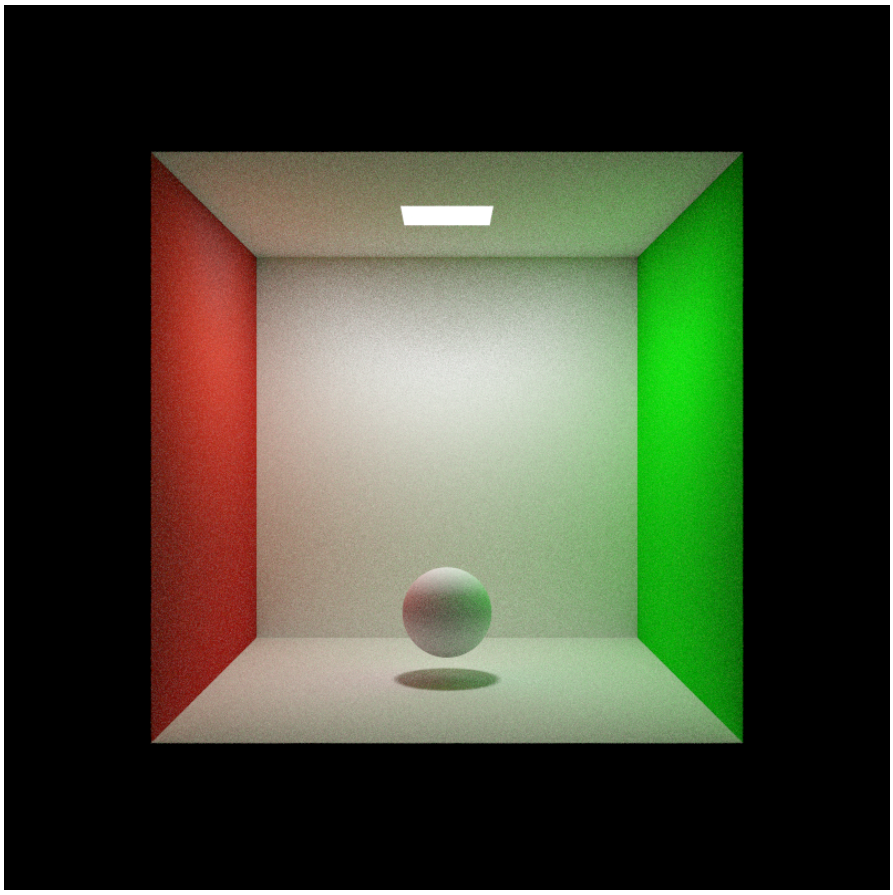


Figura 1: Ejemplo de iluminación global con transferencia de color (*color bleeding*): la luz rebota en el suelo e ilumina la parte inferior de la esfera; la luz reflejada en las paredes modula el sombreado de la esfera con el color de cada pared.

del autor: el visor existente utiliza únicamente primitivas SDF analíticas. Para trabajar con geometría poligonal arbitraria, se propone sustituir estas primitivas por texturas tridimensionales que almacenen el campo de distancias generado en GPU. Esta transición proporciona un banco de pruebas controlado para estudiar la construcción del volumen y observar cómo sus imprecisiones afectan al resultado visual.

El trabajo comparará un método base de cálculo de distancias punto-triángulo en GPU con una alternativa que combine la captura de datos de superficie mediante rasterización y la propagación espacial mediante *Jump Flooding* (JFA) [7]. Al partir de datos superficiales proyectados en lugar de evaluar cada triángulo de la malla desde cada vóxel, esta segunda aproximación no solo busca desacoplar la fase de propagación de la complejidad poligonal en gráficos, sino que también guarda paralelismo con la construcción de campos de distancia a partir de capturas parciales o sensores de profundidad en simulación física y robótica. La pregunta que orienta la evaluación es qué compromiso ofrece esta alternativa entre tiempo de generación, consumo de memoria y error de distancia respecto al método de referencia.

El producto será un prototipo técnico de generación y evaluación de SDF, acompañado de un visor interactivo, dentro del área de programación gráfica y motores de videojuegos. En el visor se podrá cargar una escena de prueba, seleccionar el campo generado por cada método, inspeccionar cortes del volumen y observar su utilización en la iluminación. La generación de los campos y su comparación constituirán el núcleo del trabajo; el visor actuará como herramienta de demostración y análisis.

1.2. Objetivos del trabajo

1.2.1. Objetivo general

Diseñar, implementar y evaluar dos métodos de generación de campos de distancia con signo en GPU a partir de mallas poligonales, comparando una alternativa basada en rasterización y propagación espacial con un método de referencia, y analizando su tiempo de generación, consumo de memoria y precisión mediante pruebas numéricas y un visor interactivo de iluminación.

1.2.2. Objetivos específicos

OE1. Adaptar el visor de partida. Integrar el prototipo previo del autor en una aplicación modular en C++ con OpenGL 4.6 y SDL3, con controles de cámara, iluminación e interfaz, para disponer de un entorno de pruebas durante el desarrollo.

OE2. Incorporar el muestreo de campos volumétricos. Sustituir las primitivas SDF analíticas por el muestreo de texturas tridimensionales de distancia, adaptando el *ray marching* para representar iluminación indirecta difusa de un rebote, oclusión ambiental y sombras suaves. El sombreado utilizará también la información de superficie necesaria.

OE3. Implementar y comprobar el método de referencia. Desarrollar un generador de distancias volumétricas en GPU mediante *compute shaders*. Antes de utilizar sus resultados como referencia, comprobar su funcionamiento con geometrías de control cuyas distancias sean conocidas.

OE4. Implementar la alternativa de generación rápida. Diseñar y desarrollar un método que combine la extracción de datos de superficie mediante rasterización de un G-Buffer con la propagación mediante JFA en tres dimensiones. Se estudiarán la cobertura de la captura, el comportamiento de la propagación y el procedimiento de determinación del signo.

OE5. Facilitar la inspección y comparación de los campos. Incorporar un selector entre los volúmenes generados por ambos métodos y modos de visualización para examinar cortes del volumen y componentes de iluminación. La visualización adicional de vóxeles tendrá menor prioridad que estas herramientas.

OE6. Ejecutar la evaluación experimental. Diseñar y realizar pruebas sobre modelos geométricos de control y las escenas seleccionadas para comparar el tiempo de generación en milisegundos, el consumo de memoria de vídeo y el error de distancia de la alternativa respecto al método base. Las condiciones de comparación y el procedimiento de medición se concretarán durante la fase de diseño.

1.2.3. Delimitación del alcance

El trabajo se centrará en dos procedimientos de generación de SDF en GPU: un método de referencia y una alternativa basada en rasterización y JFA. No se plantea desarrollar un generador universal para cualquier tipo de malla ni un motor gráfico de propósito general.

Durante la fase de especificación y diseño se definirán las condiciones de entrada de las mallas, el procedimiento de obtención del signo, la captura de datos de superficie y el protocolo experimental. La selección final de las versiones de Cornell Box y Crytek Sponza dependerá de su adecuación a estas condiciones. Los modelos geométricos de control permitirán comprobar el funcionamiento antes de abordar escenas de mayor complejidad.

El rendimiento del visor se analizará por separado del tiempo de generación del campo. La navegación y la visualización deberán permitir examinar los resultados de forma interactiva, pero no se establece como requisito inicial reconstruir todo el volumen en cada fotograma.

Ante una reducción de alcance, se conservarán los dos generadores, su comprobación numérica, la comparación experimental y una demostración básica de iluminación indirecta difusa de un rebote. La inspección por cortes tendrá prioridad sobre la visualización de vóxeles. También podrán reducirse los controles secundarios de la interfaz y los refinamientos visuales, sin sustituir con ellos el trabajo de evaluación.

1.3. Impacto en sostenibilidad, ético-social y de diversidad

1.3.1. Impacto en sostenibilidad

El proyecto estudiará el uso de recursos computacionales mediante la comparación del tiempo y la memoria necesarios para generar campos de distancia en GPU. Los resultados permitirán identificar en qué condiciones una aproximación basada en propagación resulta ventajosa y qué pérdida de precisión puede introducir.

La implementación se planteará sin depender de extensiones específicas de trazado de

rayos acelerado por hardware. No obstante, seguirá requiriendo una GPU compatible con las tecnologías seleccionadas.

El trabajo no incluirá mediciones de consumo energético ni de emisiones. Por tanto, una posible mejora del tiempo de ejecución o del uso de memoria no se presentará como una reducción demostrada del impacto ambiental.

1.3.2. Impacto ético-social

Se prevé publicar el código y la documentación para facilitar el estudio de los métodos y la reproducción de las pruebas. La documentación describirá los requisitos, las condiciones de reutilización y las limitaciones observadas.

Se mantendrá la atribución de las bibliotecas, modelos y fuentes utilizadas, así como una declaración del uso de asistentes de inteligencia artificial. La utilidad del proyecto para estudiantes y desarrolladores se considerará una contribución potencial, sin atribuirle una reducción demostrada de los costes de producción.

1.3.3. Impacto en diversidad

El proyecto no contempla recopilar datos demográficos ni realizar estudios con usuarios. Las medidas relacionadas con la diversidad se orientarán a facilitar el acceso al conocimiento mediante una documentación clara y una descripción de los requisitos técnicos.

La necesidad de disponer de una GPU compatible puede limitar el acceso al prototipo. Por ello, no se dará por supuesta su ejecución adecuada en equipos de prestaciones reducidas.

1.4. Enfoque y método seguido

El proyecto combinará un marco de investigación, un método de desarrollo del software y un sistema de seguimiento del trabajo.

1.4.1. Marco de investigación

Se utilizará la metodología de investigación en ciencias del diseño, *Design Science Research Methodology* (DSRM), propuesta por Peffers et al. [8]. Este marco organiza el trabajo alrededor de la construcción y evaluación de un artefacto tecnológico.

En este proyecto, el artefacto principal estará formado por los métodos de generación de campos de distancia en GPU. El visor servirá para demostrar su uso y examinar las consecuencias visuales de los campos obtenidos.

El proceso abarcará la identificación del problema, la definición de objetivos, el diseño y desarrollo, la demostración, la evaluación y la comunicación de resultados. La comparación experimental permitirá contrastar los métodos sin presuponer que la alternativa basada en JFA será superior en todas las métricas o escenas.

1.4.2. Metodología de producción del software

El desarrollo seguirá un enfoque incremental e iterativo basado en prototipos. Se adaptará primero el visor existente y se incorporará después el muestreo de texturas tridimensio-

nales. Para esta integración inicial se utilizarán volúmenes de prueba, sin esperar a que los generadores a partir de mallas estén terminados.

A continuación, se incorporarán el método base, la alternativa JFA y los modos de inspección. Cada incremento se comprobará con casos reducidos antes de pasar a escenas de mayor complejidad.

Las decisiones de diseño y los problemas encontrados se documentarán durante la implementación. La memoria se redactará de forma progresiva, en lugar de concentrar toda la documentación al final del proyecto.

1.4.3. Gestión del proyecto

El seguimiento se realizará mediante un tablero Personal Kanban, con tareas vinculadas a los objetivos y a los hitos de entrega. Se revisará semanalmente el avance para detectar bloqueos, actualizar prioridades y registrar las desviaciones respecto a la planificación.

Git se utilizará para el control de versiones y la trazabilidad de los cambios, siguiendo la convención *Conventional Commits*. Las revisiones con el tutor permitirán contrastar el alcance y orientar las decisiones que afecten a la viabilidad del trabajo.

1.4.4. Distribución del trabajo entre entregas

PEC 1. Definición y planificación. Se concretarán el problema, la motivación, los objetivos, el alcance y la planificación inicial. Esta entrega no incluirá nueva implementación del proyecto.

PEC 2. Estado del arte, materiales y métodos y entorno de validación. Se desarrollarán el capítulo 2 (estado del arte e hipótesis) y el capítulo 3 (estrategia de desarrollo, herramientas, protocolo experimental y entorno de pruebas). En el plano práctico, se consolidará y verificará el visor interactivo con filtrado SVGF (OE1) y se integrará el muestreo de texturas 3D mediante volúmenes de control generados por un *compute shader* preliminar (avance de OE2), despejando los riesgos de integración antes de abordar la geometría poligonal.

PEC 3. Diseño e implementación de generadores volumétricos a partir de mallas. Se redactará el capítulo 4 (diseño y arquitectura) y se implementarán la carga y empaquetado de mallas en *buffers* SSBO, el generador base de referencia punto-triángulo (OE3), el generador alternativo basado en captura de superficie y propagación JFA 3D (OE4) y el inspector de cortes bidimensionales (OE5). Se realizarán las primeras comprobaciones numéricas y mediciones comparativas.

PEC 4. Evaluación y cierre. Se completará la batería experimental, se analizarán los resultados y se redactarán las conclusiones. También se consolidará la memoria y se preparará la presentación y defensa en vídeo.

1.5. Planificación del trabajo

1.5.1. Recursos necesarios

Hardware. Se prevé utilizar una estación de trabajo con procesador multinúcleo x86-64, al menos 16 GB de memoria RAM y una tarjeta gráfica dedicada compatible con

OpenGL 4.6, *compute shaders* y texturas tridimensionales con acceso mediante *shader image load/store*. Las resoluciones de trabajo se ajustarán a la memoria disponible y a los resultados de las pruebas.

Software y entorno de desarrollo. Se utilizará GNU/Linux, un compilador compatible con C++20 —GCC o Clang—, CMake 3.20 o superior y Visual Studio Code. RenderDoc y las herramientas de monitorización de GPU servirán para inspeccionar recursos y analizar el comportamiento de la aplicación.

Bibliotecas. La aplicación utilizará SDL3, GLM, Assimp y Dear ImGui para la gestión de ventanas y entrada, las operaciones matemáticas, la importación de mallas y la interfaz.

Modelos de prueba. Se emplearán geometrías de control y las versiones seleccionadas de Cornell Box y Crytek Sponza, en formatos compatibles con el importador, como OBJ o glTF.

1.5.2. Dedicación y seguimiento

La planificación se elaborará a partir de una dedicación estimada de **20 horas semanales** (correspondiente a los 12 créditos ECTS de la asignatura, con un total estimado de 300 horas distribuidas a lo largo del semestre), que incluirá investigación, desarrollo, pruebas, documentación y revisión de las entregas.

La implementación principal se concentrará en el periodo correspondiente a la PEC 3. La documentación se mantendrá durante todas las fases, y la preparación del procedimiento de medición deberá quedar avanzada antes de comenzar la batería experimental final.

La disponibilidad durante el periodo final se tendrá en cuenta al concretar la dedicación. La entrega final de la PEC 4 se realizará el 23 de diciembre de 2026, concluyendo el trabajo antes del receso navideño. Si fuera necesario resolver incidencias durante la recta final, se ajustará el número de configuraciones experimentales secundarias, preservando la validez de la comparación principal entre los dos métodos.

El seguimiento semanal permitirá contrastar el trabajo realizado con el previsto y justificar las modificaciones del calendario.

1.5.3. Hitos por PEC

PEC 1 — 11/10/2026: definición del proyecto e introducción. Se completarán la ficha del trabajo y el capítulo 1 de la memoria, con la justificación, los objetivos, el impacto, la metodología y la planificación. También se organizarán el repositorio y la documentación inicial.

PEC 2 — 08/11/2026: estado del arte, materiales y métodos y consolidación del visor base. Se redactarán el capítulo 2 (estado del arte e hipótesis) y el capítulo 3 (estrategia de desarrollo, herramientas, protocolo experimental y entorno de validación). En el plano práctico, se consolidará y verificará el visor interactivo con reducción de ruido SVGF sobre primitivas analíticas definidas en el shader (OE1), incorporando a continuación la visualización y muestreo de texturas 3D de distancia (`GL_TEXTURE_3D`) generadas también a partir de primitivas analíticas a modo de *placeholder* (avance de OE2).

PEC 3 — 29/11/2026: diseño, implementación de generadores sobre mallas y re-

sultados intermedios. Se desarrollará el capítulo 4 (diseño y arquitectura) y se sustituirá el generador procedimental de prueba por la lectura de mallas poligonales en GPU mediante *buffers* SSBO, implementando el generador base de referencia (OE3) y la alternativa acelerada mediante rasterización y JFA 3D (OE4). Se incorporará el visor de cortes ortogonales 2D (OE5) y se conectarán los volúmenes generados con el trazador de iluminación (consolidación de OE2). La entrega incluirá las primeras comprobaciones numéricas y mediciones comparativas.

PEC 4 — 23/12/2026: resultados, memoria final y defensa. Se completarán las pruebas, el análisis del capítulo 5 y las conclusiones del capítulo 6. Se revisará el diseño documentado y se consolidará la memoria. La presentación y la grabación de la defensa se prepararán antes del margen final de contingencia.

1.5.4. Cronograma

La Tabla 1 recoge la planificación actualizada del proyecto. Durante la fase correspondiente a la PEC 2 se ha adelantado la consolidación del visor interactivo (OE1) y la transición inicial a texturas 3D mediante volúmenes de prueba generados en GPU a partir de primitivas analíticas (OE2 preliminar). Esta adaptación temprana mitiga el riesgo de integración visual (R2) y permite dedicar íntegramente el bloque de la PEC 3 a la implementación y validación de los dos generadores a partir de mallas poligonales (OE3 y OE4) y al inspector de cortes (OE5).

Tabla 1: Cronograma del proyecto.

Fase	Actividad	Inicio	Fin
PEC 1	Definición del proyecto y ficha del trabajo	23/09/2026	06/10/2026
PEC 1	Redacción del capítulo 1 y preparación de la entrega	06/10/2026	11/10/2026
PEC 2	Investigación y redacción del estado del arte	12/10/2026	25/10/2026
PEC 2	Consolidación del visor con SVGF sobre primitivas analíticas (OE1)	15/10/2026	28/10/2026
PEC 2	Estrategia metodológica, herramientas y protocolo de evaluación	23/10/2026	01/11/2026
PEC 2	Integración de texturas 3D SDF *placeholder* y entrega PEC 2 (OE2 preliminar)	28/10/2026	08/11/2026
PEC 3	Diseño arquitectónico, empaquetado SSBO e implementación del generador base (OE3)	09/11/2026	16/11/2026
PEC 3	Implementación del generador alternativo por captura superficial y JFA 3D (OE4)	15/11/2026	23/11/2026
PEC 3	Implementación del visor de cortes 2D y conexión de mallas con el trazador (OE5)	21/11/2026	26/11/2026
PEC 3	Comprobaciones numéricas iniciales, primeras mediciones y entrega PEC 3	24/11/2026	29/11/2026
PEC 4	Batería experimental final y recogida de datos	30/11/2026	08/12/2026
PEC 4	Análisis de resultados y discusión	07/12/2026	14/12/2026
PEC 4	Revisión del diseño, conclusiones y consolidación de la memoria	12/12/2026	18/12/2026
PEC 4	Preparación de la presentación y grabación de la defensa	17/12/2026	20/12/2026
PEC 4	Margen de contingencia y comprobación final de los entregables	21/12/2026	23/12/2026

La entrega final está prevista para el 23 de diciembre de 2026. El margen de contingen-

cia no se considerará tiempo disponible para añadir funcionalidades nuevas, sino una reserva para resolver incidencias y corregir los entregables.

1.5.5. Riesgos y medidas de mitigación

La Tabla 2 recoge los principales riesgos técnicos y organizativos. Las valoraciones de probabilidad e impacto son iniciales y se revisarán conforme avance el proyecto.

Tabla 2: Riesgos y estrategias de mitigación.

ID	Riesgo	Prob.	Impacto	Mitigación y contingencia
R1	Consumo excesivo de memoria de vídeo por los volúmenes y las estructuras auxiliares.	Media	Alto	Parametrizar la resolución, utilizando inicialmente tamaños exploratorios como 64^3, 128^3 y 256^3. Estudiar formatos como `GL_R16F` y `GL_R32F` y contabilizar también los recursos temporales utilizados durante la generación.
R2	Rendimiento insuficiente para utilizar el visor de forma interactiva.	Media	Medio	Medir por separado el renderizado y la generación del campo. Ajustar el número de pasos de trazado, los criterios de parada y la resolución de cálculo de la iluminación indirecta.
R3	Cobertura incompleta de la geometría, errores de distancia o fallos en la determinación del signo.	Alta	Alto	Comenzar con mallas de control simples y cerradas. Examinar por separado la captura, la distancia y el signo, y distinguir los errores del campo de los artefactos introducidos por el renderizado. Concretar las correcciones durante el diseño y la implementación.
R4	Dificultades en la implementación de JFA en tres dimensiones.	Media	Alto	Desarrollar y comprobar la propagación en casos reducidos antes de integrarla con escenas completas. Contrastar los resultados esperados para distinguir los fallos de implementación del error de aproximación del método.
R5	Retrasos respecto a los hitos de entrega.	Media	Alto	Revisar semanalmente el avance. Proteger los dos generadores, las comprobaciones numéricas, la comparación experimental y la demostración básica de iluminación. Reducir primero la visualización de vóxeles, los controles secundarios y los refinamientos visuales.

Las medidas de contingencia darán prioridad a obtener resultados comparables y documentados. Una demostración visual más completa no compensará la ausencia de comprobaciones o de datos suficientes para evaluar los métodos.

1.6. Breve sumario de productos obtenidos

El desarrollo del proyecto dará lugar a cuatro resultados principales, cuya descripción detallada se aborda en el resto de capítulos de la memoria:

- **Módulos de generación de SDF en GPU:** un método base de cálculo exacto de distancias punto-triángulo mediante *compute shaders* y una alternativa acelerada que combina la captura de superficie por rasterización con la propagación espacial

mediante JFA en tres dimensiones.

- **Aplicación interactiva de validación e inspección:** un visor modular en C++20 y OpenGL 4.6 con *path tracing* sobre campos SDF, reducción de ruido espacio-temporal SVGF, muestreo de texturas tridimensionales y un inspector de cortes ortogonales bidimensionales (*slice viewer*).
- **Banco de pruebas y conjunto de datos experimentales:** una batería de evaluación sobre geometrías de control y escenas de complejidad creciente que cuantificará tiempos de generación en GPU, consumo de memoria de vídeo y error numérico de distancia respecto al método de referencia.
- **Memoria técnica y defensa del trabajo:** la documentación académica completa conforme a la normativa de la asignatura, junto con la presentación y defensa audiovisual del prototipo.

1.7. Breve descripción de los otros capítulos de la memoria

1.7.1. Capítulo 2. Estado del arte

Revisa los fundamentos matemáticos de los campos de distancia con signo (SDF) y el *sphere tracing*, la formulación de distancia mínima punto-triángulo en GPU, los métodos topológicos de determinación del signo y el algoritmo *Jump Flooding* (JFA) en 3D. Asimismo, contextualiza la iluminación global interactiva y la reconstrucción espacio-temporal (SVGF), cerrando con la síntesis de aportaciones y las hipótesis del trabajo.

1.7.2. Capítulo 3. Materiales y métodos

Expone la estrategia de diseño y desarrollo adoptada, analizando las alternativas consideradas y los criterios de decisión. Describe la pila tecnológica y las herramientas empleadas, detalla el entorno interactivo de validación ya implementado (Hito OE1 y avance de OE2) y formaliza las escenas de prueba y el protocolo experimental.

1.7.3. Capítulo 4. Diseño

Describirá los requisitos del sistema y la arquitectura de los módulos generadores de SDF en GPU y del inspector volumétrico, incluyendo el empaquetado de geometrías en *buffers* SSBO, la organización de los *compute shaders*, la captura de superficie, el cálculo del signo y el esquema *ping-pong* de propagación.

1.7.4. Capítulo 5. Resultados

Presentará y analizará las mediciones obtenidas sobre los modelos de control y las escenas seleccionadas. Comparará ambos métodos en tiempo de generación, consumo de VRAM y error numérico de distancia, relacionando las discrepancias métricas con su impacto visual en la iluminación global.

1.7.5. Capítulo 6. Conclusiones y trabajo futuro

Contrastará los resultados alcanzados con los objetivos e hipótesis iniciales, evaluará el seguimiento de la planificación y de los impactos previstos, y recogerá las limitaciones observadas junto con las líneas de extensión futura.

1.7.6. Capítulo 7. Glosario

Define los términos técnicos y acrónimos más relevantes utilizados a lo largo de la memoria.

1.7.7. Capítulo 8. Bibliografía

Recoge las fuentes académicas y técnicas citadas en el documento mediante numeración correlativa estandarizada.

1.7.8. Capítulo 9. Anexos

Incluirá el material complementario autocontenido necesario para consultar o reproducir el trabajo al cierre del proyecto.

2. Estado del arte

2.1. Campos de distancia con signo (SDF) y *sphere tracing*

Un campo de distancia con signo (*Signed Distance Field*, SDF) es una función escalar $f : \mathbb{R}^3 \to \mathbb{R}$ que asigna a cada coordenada espacial la distancia euclídea mínima a la superficie delimitadora más próxima $\partial\mathcal{S}$ de la escena:

$$f(p) = \text{sign}(p) \cdot \min_{q \in \partial\mathcal{S}} \|p - q\| \tag{1}$$

El signo positivo identifica los puntos pertenecientes al exterior del volumen, el valor cero define la superficie geométrica $\partial\mathcal{S}$, y el signo negativo determina el interior de cuerpos cerrados y orientables. La propiedad matemática fundamental de un campo de distancia continuo es que satisface la ecuación *eikonal* $\|\nabla f(p)\| = 1$ en casi todo el dominio continuo. Esto garantiza que la magnitud del gradiente espacial coincide con la unidad y que su vector normalizado corresponde con el vector normal unitario a la superficie:

$$n = \nabla f(p) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right) \tag{2}$$

En simulación física y robótica, esta relación proporciona de forma directa la profundidad de interpenetración ($f(p) < 0$), el margen de seguridad frente a obstáculos ($f(p) > 0$) y la dirección normal de contacto o repulsión ($\nabla f(p)$) [2, 3]. En síntesis de imagen, esta misma propiedad fundamenta el algoritmo de *sphere tracing* introducido por Hart [4]. A diferencia del trazado por pasos uniformes (*ray marching* regular), el *sphere tracing* garantiza que dentro de una esfera de radio $f(p)$ centrada en la posición actual del rayo no existe intersección geométrica alguna con la escena. Por tanto, el avance del rayo puede adaptarse dinámicamente a la distancia devuelta por el campo:

$$p_{k+1} = p_k + f(p_k) \cdot \omega \tag{3}$$

Esta aceleración reduce drásticamente el número de evaluaciones necesarias para encontrar una intersección, facilitando tanto el cálculo de sombras suaves cónicas mediante la monitorización del cociente $\min(f(p)/t)$ como el cálculo de rebotes indirectos difusos y oclusión ambiental. En la industria del videojuego, motores de última generación como Unreal Engine 5 con su subsistema Lumen [6] emplean campos de distancia discretizados para realizar consultas de oclusión e iluminación indirecta independientes de la aceleración por hardware dedicada.

2.2. Generación de SDF volumétricos en GPU a partir de mallas poligonales

A diferencia de las primitivas analíticas (esferas, cilindros, cajas) cuyas funciones SDF se evalúan mediante expresiones algebraicas cerradas de coste constante $\mathcal{O}(1)$, las mallas poligonales arbitrarias compuestas por N_{tris} triángulos requieren procedimientos de discretización espacial sobre cuadrículas volumétricas tridimensionales de resolución

$V_x \times V_y \times V_z = V^3$ vóxeles.

2.2.1. Formulación matemática de la distancia mínima punto-triángulo en $\mathbb{R}^3$

Dado un punto de consulta $p \in \mathbb{R}^3$ (correspondiente al centro espacial de un vóxel) y un triángulo tridimensional T definido por sus tres vértices ordenados $\{v_0, v_1, v_2\}$, los vectores directores de sus aristas se expresan como $e_0 = v_1 - v_0$ y $e_1 = v_2 - v_0$. El vector normal no normalizado al plano que contiene al triángulo viene dado por el producto vectorial $n_T = e_0 \times e_1$.

El cálculo de la distancia euclídea mínima entre p y el triángulo T, denotada como $d(p, T) = \text{mín}_{q \in T} \|p - q\|$, no puede resolverse simplemente calculando la distancia ortogonal al plano infinito que lo contiene, ya que la proyección ortogonal del punto puede quedar fuera de los límites geométricos del triángulo [1]. Siguiendo la formulación geométrica de regiones de Voronoi planares, el triángulo delimita siete regiones disjuntas en el espacio: la cara interior, tres aristas y tres vértices.

La parametrización de cualquier punto sobre el plano del triángulo se define en coordenadas baricéntricas o afines como:

$$q(u, v) = v_0 + u\, e_0 + v\, e_1, \quad \text{con } u \geq 0,\ v \geq 0,\ u + v \leq 1 \tag{4}$$

Definiendo el vector de desplazamiento $w = v_0 - p$, la distancia al cuadrado entre p y cualquier punto $q(u, v)$ constituye una función cuadrática:

$$\phi(u, v) = \|q(u, v) - p\|^2 = au^2 + 2buv + cv^2 + 2du + 2ev + f \tag{5}$$

donde los coeficientes escalares resultan de los productos escalares fundamentales: $a = e_0 \cdot e_0$, $b = e_0 \cdot e_1$, $c = e_1 \cdot e_1$, $d = e_0 \cdot w$, $e = e_1 \cdot w$ y $f = w \cdot w$. El mínimo global no restringido de esta parábola sobre el plano infinito ocurre en el punto crítico donde el gradiente se anula:

$$u^* = \frac{be - cd}{ac - b^2}, \quad v^* = \frac{bd - ae}{ac - b^2} \tag{6}$$

A partir de este punto crítico, se discriminan los casos geométricos:

1. **Región de cara:** Si $u^* \geq 0$, $v^* \geq 0$ y $u^* + v^* \leq 1$, la proyección ortogonal de p recae estrictamente dentro del triángulo. El punto más próximo es $q = q(u^*, v^*)$ y la distancia mínima coincide con la distancia perpendicular al plano: $d(p, T) = |(p - v_0) \cdot (n_T / \|n_T\|)|$.
2. **Regiones de aristas (3 regiones):** Si la proyección cae fuera de una de las aristas del triángulo pero dentro de los planos delimitadores de sus vértices extremos, el punto más cercano se ubica sobre el segmento de línea que define dicha arista. La solución se reduce a la proyección unidimensional acotada con parámetro $t \in [0, 1]$ sobre el segmento correspondiente.

3. **Regiones de vértices (3 regiones):** Si la proyección cae en el cono exterior de Voronoi adyacente a uno de los vértices, el punto más cercano es dicho vértice v_i, cumpliéndose $d(p,T) = \|p - v_i\|$.

2.2.2. Paralelización masiva en GPU mediante Compute Shaders

Para generar un volumen de distancias sobre una cuadrícula regular de V^3 vóxeles inscrita en una *bounding box* alineada con los ejes (*Axis-Aligned Bounding Box*, AABB), el método directo o de fuerza bruta asigna un hilo de ejecución independiente a cada vóxel (i,j,k). La posición tridimensional del centro del vóxel en espacio de mundo se calcula como:

$$p_{ijk} = \text{AABB}_{\text{mín}} + \left(\frac{(i,j,k) + 0{,}5}{V}\right) \odot (\text{AABB}_{\text{máx}} - \text{AABB}_{\text{mín}}) \tag{7}$$

En una implementación directa, cada hilo recorre secuencialmente los N_{tris} triángulos de la malla poligonal, almacenados en un *buffer* de almacenamiento en GPU (*Shader Storage Buffer Object*, SSBO), evaluando la distancia mínima y conservando el valor escalar más reducido:

$$d(p_{ijk}) = \min_{m \in \{1,\ldots,N_{\text{tris}}\}} d(p_{ijk}, T_m) \tag{8}$$

La complejidad computacional global de este enfoque es de $\mathcal{O}(V^3 \cdot N_{\text{tris}})$. Para resoluciones típicas como 128^3 ($2{,}097{,}152$ vóxeles) y mallas con $250{,}000$ triángulos, el algoritmo exige ejecutar más de 5×10^{11} pruebas punto-triángulo. Aunque esta carga computacional limita la interactividad en mallas complejas, su naturaleza masivamente paralela y exenta de divergencias de ramas complejas lo convierte en el método de referencia determinista óptimo para verificar la corrección geométrica de otros algoritmos aproximados.

2.2.3. Determinación del signo: mallas cerradas frente a mallas arbitrarias

Asignar un signo escalar a la distancia euclídea ($+d$ para el exterior y $-d$ para el interior) requiere determinar la topología del objeto. En computación gráfica se distinguen fundamentalmente dos escenarios:

1. **Mallas *manifold* cerradas (*watertight* / *2-manifold*):** Objetos perfectamente estancos, orientables, sin auto-intersecciones ni aristas compartidas por más o menos de dos caras poligonales. En estas geometrías existe una frontera inequívoca entre el volumen interno y el espacio exterior.
2. **Mallas abiertas o *non-manifold* (*open boundaries* / *polygon soup*):** Modelos comunes en entornos interactivos que contienen planos de grosor nulo, huecos geométricos, caras interiores parásitas o discontinuidades en las normales de los vértices.

Para discernir la pertenencia interior/exterior sobre la cuadrícula volumétrica se estudian tres estrategias principales en la literatura:

- **Pseudo-normales ponderadas por ángulo (*Angle-Weighted Pseudonormals*):**

Introducido por Bærentzen y Aanæs [9], este método asocia a cada primitiva geométrica (caras, aristas y vértices) un vector normal regularizado. Para una cara, se emplea su normal geométrica n_F. Para un vértice v, la pseudo-normal n_v se define como la suma ponderada de las normales de las caras adyacentes multiplicadas por el ángulo interior de apertura θ_k de cada cara en dicho vértice:

$$n_v = \sum_{k \in \mathcal{F}(v)} \theta_k \, n_{F_k} \tag{9}$$

De igual forma, para una arista compartida por dos caras se suman sus normales ponderadas. Cuando un vóxel determina cuál es la característica más cercana (cara, arista o vértice), el signo se calcula mediante el producto escalar entre el vector de desplazamiento al punto más cercano q y la pseudo-normal asociada n_q:

$$\text{sign}(p) = \text{sign}\left((p - q) \cdot n_q\right) \tag{10}$$

Este método es sumamente eficiente y de cómputo puramente local, pero resulta vulnerable si la malla no es estrictamente estanca o presenta caras invertidas, provocando inversiones de signo espurias que se propagan a través de las aperturas.

- **Conteo de intersecciones por trazado de rayos (*Ray Casting Parity Count*):** Basado en el teorema de la curva de Jordan generalizado a $\mathbb{R}^3$, se traza un rayo en una dirección arbitraria (por ejemplo, a lo largo del eje $+X$) desde el vóxel p hacia el infinito. Si el número de intersecciones con los triángulos de la escena es impar, el punto es interior; si es par, es exterior. Su mayor inconveniente en GPU radica en el elevado coste de trazar V^3 rayos y en la inestabilidad numérica producida cuando un rayo incide tangencialmente sobre una arista o vértice compartido, además de fallar categóricamente en mallas abiertas con huecos.

- ***Generalized Winding Numbers*:** Propuesto por Jacobson et al. [10], generaliza el concepto topológico de *winding number* a mallas poligonales arbitrarias con agujeros, auto-intersecciones y ruido geométrico. El *winding number* $w(p)$ en un punto p se define como la integral del ángulo sólido subtendido por la superficie orientada:

$$w(p) = \frac{1}{4\pi} \sum_{m=1}^{N_{\text{tris}}} \Omega_m(p) \tag{11}$$

donde $\Omega_m(p)$ representa el ángulo sólido con signo subtendido por el triángulo m visto desde p. Para mallas estancas, $w(p) = 1$ en el interior y $w(p) = 0$ en el exterior. En mallas imperfectas con bordes abiertos o geometrías desconectadas, $w(p)$ adopta valores intermedios que decaen suavemente, permitiendo clasificar el interior mediante un umbral de decisión robusto (típicamente $w(p) \geq 0{,}5$). Su evaluación directa requiere una complejidad $\mathcal{O}(V^3 \cdot N_{\text{tris}})$, requiriendo aproximaciones jerárquicas tipo Barnes-Hut para su aceleración.

2.3. Propagación espacial mediante Jump Flooding Algorithm (JFA) en 3D

Para superar el cuello de botella computacional $\mathcal{O}(V^3 \cdot N_{\text{tris}})$ inherente a los métodos de cálculo directo, se recurre a técnicas de propagación espacial en cuadrículas regulares. El algoritmo *Jump Flooding Algorithm* (JFA), formulado originalmente por Rong y Tan [7], permite calcular transformadas de distancia euclídea discretas (*Discrete Euclidean Distance Transform*, EDT) y aproximaciones de diagramas de Voronoi en arquitecturas masivamente paralelas con una complejidad temporal de $\mathcal{O}(V^3 \log_2 V)$, totalmente independiente del número de triángulos originales de la malla.

2.3.1. Estructura y fases del *pipeline* JFA en tres dimensiones

La adaptación de JFA a un volumen tridimensional se estructura en tres etapas secuenciales de procesamiento en GPU:

1. **Extracción e inicialización de semillas (*Seed Initialization*):** Se analiza la superficie de la malla poligonal sobre la cuadrícula tridimensional mediante rasterización volumétrica conservativa o mediante voxelización superficial directa en *compute shaders*. Para cada vóxel que contenga o intercepte un elemento de superficie, se registra su posición espacial tridimensional en una textura volumétrica de semillas (`GL_RGBA32F` o `GL_RGBA16F`). Los vóxeles vacíos que no intersecan la superficie se inicializan con un valor nulo o un valor centinela (*sentinel value*) fuera del rango de coordenadas (por ejemplo, $(-1, -1, -1)$).

2. **Pasadas iterativas de salto (*Jump Passes*):** Para una resolución cúbica de lado V (donde V es una potencia de dos, tal como 64, 128 o 256), el algoritmo ejecuta $M = \log_2 V$ pasadas sucesivas. En cada iteración $s \in \{0, 1, \ldots, M-1\}$, se define una longitud de paso o salto k que se reduce a la mitad en cada paso:

$$k = \frac{V}{2^{s+1}} \in \left\{ \frac{V}{2}, \frac{V}{4}, \ldots, 2, 1 \right\} \tag{12}$$

   En cada pasada, cada vóxel $p = (x, y, z)$ del volumen consulta la información de semilla contenida en sus 26 vecinos tridimensionales más su propia posición ($3 \times 3 \times 3 = 27$ muestras en total), desplazados a una distancia espacial de k vóxeles:

$$\mathcal{N}_k(p) = \{p + k \cdot (\delta_x, \delta_y, \delta_z) \mid \delta_x, \delta_y, \delta_z \in \{-1, 0, 1\}\} \tag{13}$$

   Para cada vecino $q \in \mathcal{N}_k(p)$ dentro de los límites del volumen, si q almacena una semilla válida $S(q)$, se evalúa la distancia euclídea $\|p - S(q)\|$. El hilo actual selecciona la semilla S^* que minimice la distancia euclídea al punto p y la escribe en la textura de salida. La sincronización se garantiza mediante un esquema de doble *buffer* (*ping-pong*), alternando las texturas de lectura y escritura entre pasadas.

3. **Cálculo de distancia final y asignación de signo:** Tras concluir la pasada final con longitud de salto unitaria ($k = 1$), cada vóxel p dispone de las coordenadas de la semilla superficial más cercana que ha logrado propagarse hasta su posición. En una pasada de finalización, se computa la distancia euclídea $\|p - S^*(p)\| \cdot \Delta x_{\text{vóxel}}$ y se le asocia el signo correspondiente obtenido del módulo de determinación topológica, almacenando la distancia final en una textura escalar volumétrica (`GL_R32F`

o `GL_R16F`).

2.3.2. Análisis de errores de aproximación y variantes (JFA $+1$, $1+$ JFA)

Aunque JFA exhibe un rendimiento sobresaliente en GPU, no calcula un diagrama de Voronoi estrictamente exacto debido a la naturaleza discreta del muestreo con pasos grandes. En ciertas configuraciones geométricas cóncavas o con características diagonales complejas, una semilla óptima puede quedar "oculta" o eclipsada detrás de semillas intermedias durante las fases de saltos largos ($k = V/2, V/4$), impidiendo que su información alcance a vóxeles que formalmente pertenecen a su celda de Voronoi.

Rong y Tan [7] identificaron y cuantificaron esta tasa de error en espacios discretos. Para mitigar estas imprecisiones sin renunciar a la velocidad en GPU, propusieron variantes de compensación:

- **Variante JFA $+1$:** Consiste en ejecutar una pasada adicional con tamaño de salto $k = 1$ inmediatamente después de finalizar la secuencia regular. Esta iteración suplementaria permite a los vóxeles corregir discontinuidades locales inmediatas en los bordes de Voronoi, reduciendo el error a una fracción casi nula del volumen total.
- **Variante $1+$ JFA:** Consiste en anteponer una pasada con salto $k = 1$ al inicio de la secuencia logarítmica (antes de $k = V/2$), garantizando que las semillas adyacentes de alta curvatura formen un frente continuo antes de proyectarse a grandes distancias.

En el contexto de este proyecto, la cuantificación experimental de este error de propagación frente al método exacto de *compute shaders* constituye uno de los núcleos de investigación analítica del TFM.

2.4. Iluminación global en tiempo real y reconstrucción espacio-temporal

Para evaluar de forma cualitativa y cuantitativa la fidelidad de los campos de distancia con signo generados en GPU, es indispensable observar su comportamiento en consultas continuas de trazado de rayos aplicadas a iluminación global interactiva. La ecuación de renderizado [5] formaliza el equilibrio energético de la luz en un entorno virtual, definiendo la radiancia saliente $L_o(p, \omega_o)$ desde un punto p hacia la dirección ω_o como la suma de la radiancia propia emitida L_e y la integral de la radiancia incidente L_i modulada por la función de distribución de reflectancia bidireccional (BRDF) f_r y el factor geométrico del coseno:

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o)\, L_i(p, \omega_i)\, (\omega_i \cdot n)\, d\omega_i \tag{14}$$

En aplicaciones interactivas a frecuencias de refresco elevadas (60 a 120 FPS), la evaluación estocástica de esta integral mediante *path tracing* está restringida a un presupuesto computacional mínimo de una única muestra por píxel (1 SPP, *Sample Per Pixel*). El muestreo a 1 SPP produce una imagen dominada por una varianza estocástica extrema (ruido blanco o granular de alta frecuencia). Si se pretendiera evaluar visualmente el impacto de las discontinuidades geométricas o los errores de un campo SDF sobre una

imagen bruta a 1 SPP, el ruido estocástico de Monte Carlo enmascararía por completo cualquier artefacto sutil introducido por la discretización del volumen.

Por esta razón, la arquitectura del TFM integra un *pipeline* de reconstrucción espacio-temporal robusto basado en el algoritmo SVGF (*Spatiotemporal Variance-Guided Filtering*) [11] que filtra la señal luminosa preservando los bordes geométricos mediante cuatro etapas sinérgicas:

1. **Demodulación de albedo:** Desacopla la reflectancia de los materiales respecto a la irradiancia incidente pura ($E = L/\,\text{máx}(\rho, \epsilon)$), permitiendo que el filtro *wavelet* suavice gradientes luminosos sin difuminar los detalles texturales del material.
2. **Reproyección temporal:** Rastrea los píxeles a través de matrices de transformación históricas y vectores de velocidad, acumulando muestras coherentes a lo largo de fotogramas sucesivos para elevar exponencialmente el recuento efectivo de muestras.
3. **Estimación y seguimiento de varianza:** Evalúa continuamente los momentos estadísticos de la señal luminosa ($\mu_1 = \mathbb{E}[L]$ y $\mu_2 = \mathbb{E}[L^2]$), computando la varianza local $\sigma^2 = \mu_2 - \mu_1^2$ para adaptar dinámicamente la agresividad del filtrado.
4. **Filtrado multiescala À-Trous:** Aplica un filtro espacial separable basado en *wavelets* [12] sobre un núcleo B_3-spline de 5×5 en múltiples pasadas *ping-pong* con saltos exponenciales crecientes (2^i píxeles), ponderado por funciones de preservación de bordes (*edge-stopping functions*) que detectan discontinuidades en normales, distancias planares y varianza local.

Este *pipeline* de reconstrucción espacio-temporal permite evaluar en tiempo real tanto funciones de distancia analíticas codificadas directamente en el shader como texturas volumétricas tridimensionales (`GL_TEXTURE_3D`), desacoplando el análisis de fidelidad del campo de distancias respecto al ruido estocástico propio del muestreo de Montecarlo a baja densidad.

2.5. Síntesis, aportación e hipótesis del trabajo

La revisión de la literatura pone de manifiesto una tensión fundamental en la generación de campos de distancia volumétricos a partir de mallas poligonales en GPU. Por un lado, la evaluación directa de la distancia mínima punto-triángulo garantiza exactitud geométrica sobre la cuadrícula discreta, pero presenta una complejidad $\mathcal{O}(V^3 \cdot N_{\text{tris}})$ que penaliza severamente el tiempo de cómputo a medida que aumenta la densidad poligonal. Por otro lado, los métodos de propagación espacial como el algoritmo *Jump Flooding* (JFA) en tres dimensiones reducen el coste de propagación a $\mathcal{O}(V^3 \log_2 V)$, independizándolo del número de triángulos de la malla original, pero introducen errores de aproximación derivados tanto de la discretización inicial de las semillas en la superficie como de las pasadas de salto en el diagrama de Voronoi discreto [7].

La aportación principal de este trabajo consiste en diseñar, implementar y comparar experimentalmente ambos enfoques bajo condiciones idénticas de hardware y resolución volumétrica, evaluando no solo la discrepancia numérica del campo generado respecto a la solución de referencia, sino también cómo afectan dichas imprecisiones a un caso de

uso sensible a los errores del campo como es la iluminación global interactiva por *sphere tracing*. Asimismo, el estudio del generador basado en captura superficial y propagación JFA 3D ofrece una base metodológica extensible a escenarios donde la geometría de entrada se obtiene como proyecciones de profundidad.

A partir de este análisis se formulan las siguientes hipótesis de trabajo que guiarán la evaluación experimental:

- **H1 (Escalabilidad temporal):** La alternativa basada en captura superficial y propagación mediante JFA 3D presentará un tiempo de generación en GPU significativamente inferior al del método exacto punto-triángulo cuando la complejidad geométrica de la escena sea elevada ($N_{\text{tris}} \gg 10^3$), manteniéndose el coste de las pasadas de propagación prácticamente constante para una resolución volumétrica fija V^3.
- **H2 (Compromiso entre precisión y consumo de memoria):** El campo generado mediante JFA 3D presentará un error numérico acotado (MAE y RMSE inferiores al tamaño métrico de un vóxel en la mayor parte del dominio) a cambio de un mayor requerimiento temporal de memoria de vídeo durante la generación debido a las texturas auxiliares del esquema *ping-pong*.
- **H3 (Viabilidad en consultas espaciales de iluminación):** Para resoluciones volumétricas medias y altas (128^3 y 256^3), las desviaciones de distancia introducidas por JFA 3D no degradarán de forma crítica la iluminación indirecta difusa de un rebote ni la oclusión ambiental al combinarse con un *surface bias* adaptado al tamaño de vóxel, concentrándose las diferencias visuales perceptibles en las penumbras de contacto y en las cavidades geométricas finas.

3. Materiales y métodos

3.1. Estrategia de desarrollo, alternativas y decisiones técnicas

Siguiendo la metodología de investigación en ciencias del diseño (DSRM) [8] y el enfoque incremental descritos en el Capítulo 1, el desarrollo se articula en torno a decisiones técnicas que acotan las alternativas existentes en la literatura a los objetivos del trabajo:

1. **Elección del método de referencia frente a estructuras jerárquicas en CPU:** Para obtener un volumen de distancias exacto con el que contrastar la alternativa aproximada, se contemplan dos opciones: construir una jerarquía de volúmenes envolventes (*Bounding Volume Hierarchy*, BVH) en CPU o evaluar directamente los triángulos en GPU mediante *compute shaders* leyendo desde un *buffer* SSBO. Se opta por la evaluación directa masivamente paralela en GPU porque evita la complejidad de mantener y recorrer árboles espaciales con divergencia de ramas en shaders, aprovecha el ancho de banda de memoria de la tarjeta gráfica y garantiza una referencia determinista exacta punto-triángulo en cada centro de vóxel. Antes de emplearlo como referencia sobre escenas complejas, su exactitud se verifica frente a geometrías de control (esfera y cubo) cuyas funciones SDF analíticas son conocidas en forma cerrada.

2. **Diseño de la alternativa rápida (captura superficial + JFA 3D):** Para desacoplar la generación del campo del número de triángulos de la malla, se elige combinar una primera fase de extracción de semillas superficiales con una segunda fase de propagación mediante JFA en tres dimensiones ($M = \log_2 V$ pasadas con saltos $k = V/2^{s+1}$ sobre los 26 vecinos tridimensionales y esquema de doble *buffer ping-pong*). Se incluyen también en el estudio las variantes JFA $+\,1$ y $1 +$ JFA [7] para cuantificar si el coste de una pasada adicional a distancia unitaria compensa la reducción del error en las fronteras de Voronoi.

3. **Estrategia de determinación del signo interior/exterior:** Entre las tres familias revisadas en el estado del arte (pseudo-normales ponderadas por ángulo [9], conteo de intersecciones por rayos y *Generalized Winding Numbers* [10]), se prioriza el trabajo con mallas cerradas o con orientación consistente de caras, combinando la información normal/topológica local con pruebas de paridad o propagación para evitar el coste cuadrático de evaluar ángulos sólidos completos en cada vóxel.

4. **Aislamiento y validación progresiva del entorno visual (Hito OE1 y avance OE2):** Un riesgo metodológico habitual al evaluar estructuras geométricas mediante renderizado es confundir los artefactos propios del trazador de rayos o del filtro de ruido con los errores del campo de distancias. Por ello, esta primera fase del visor se estructura en dos pasos incrementales: primero, inyectando primitivas analíticas directamente en la función de distancias del shader para validar el trazador por *sphere tracing* y el filtro SVGF sin errores de discretización; y segundo, añadiendo la visualización de texturas de distancia 3D (`GL_TEXTURE_3D`) generadas también con primitivas analíticas codificadas directamente en un *compute shader* a modo de *placeholder*, listas para ser sustituidas por las texturas generadas a partir de mallas poligonales en las fases posteriores.

3.2. Herramientas y pila tecnológica

Para garantizar la portabilidad, el control explícito sobre la memoria de vídeo y la ausencia de dependencias de hardware propietario de trazado de rayos, se selecciona una pila basada en estándares abiertos encabezada por C++20 y OpenGL 4.6 Core Profile. OpenGL 4.6 proporciona soporte nativo para *compute shaders*, acceso aleatorio de lectura y escritura a texturas tridimensionales mediante `glBindImageTexture` (`shader_image_load_store`), *buffers* de almacenamiento masivo SSBO con alineamiento `std430` y consultas asíncronas de tiempo en GPU (`GL_TIME_ELAPSED`). La Tabla 3 resume los componentes y herramientas utilizados.

Tabla 3: Pila tecnológica y entorno de desarrollo.

Componente	Versión / Especificación	Propósito en el proyecto
Lenguaje de programación	C++20 (ISO/IEC 14882:2020)	Núcleo de la aplicación, gestión de recursos y coordinación del *pipeline* en GPU.
API Gráfica	OpenGL 4.6 Core Profile / GLSL 4.60	Ejecución de *compute shaders*, FBOs, texturas 3D, SSBOs y *deferred rendering*.
SDL3	Versión 3.2.0	Creación de ventanas, contexto OpenGL, captura de eventos y temporización.
GLAD	Especificación Core 4.6	Carga dinámica de punteros a funciones de OpenGL 4.6 en el host.
GLM	Versión 1.0.1	Álgebra lineal tridimensional, matrices de transformación y vectores.
Assimp	Versión 5.4+	Importación de mallas tridimensionales (OBJ, glTF), cálculo de normales y AABB.
Dear ImGui	Versión 1.91.8	Interfaz gráfica para control de parámetros, diagnóstico e inspección de volúmenes.
stb_image / stb_image_write	Versión 2.30 / 1.16	Carga de recursos auxiliares y exportación de capturas sin pérdida.
RenderDoc	Versión 1.3x	Inspección de texturas 3D, SSBOs, estados del pipeline y depuración de shaders.
Sistema de compilación	CMake 3.28+ con Ninja	Configuración multiplataforma y compilación incremental.
Compilador C++	GCC 14.2 / Clang 18.1	Generación de código binario nativo optimizado.
Hardware de pruebas	AMD Radeon RX 9060 XT (RDNA 4) / Mesa 26.2	Plataforma principal de desarrollo y evaluación en GPU.

3.3. Entorno interactivo de validación desarrollado (Hito OE1 y avance OE2)

Como parte del trabajo realizado hasta esta entrega, se ha implementado y verificado la primera fase del visor interactivo que actuará como banco de pruebas experimental durante todo el proyecto. El desarrollo de este entorno se articula de forma incremental en dos etapas sucesivas: en primer lugar, se implementan primitivas analíticas inyectadas directamente en la función de evaluación de distancias del shader para construir y validar

el *path tracer* por *sphere tracing* a 1 muestra por píxel (1 SPP) y la reconstrucción espacio-temporal mediante SVGF [11]; en segundo lugar, se incorpora el soporte y la visualización de texturas tridimensionales de distancia (`GL_TEXTURE_3D`), generadas inicialmente mediante primitivas analíticas codificadas también de forma directa en un *compute shader* a modo de *placeholder*, con el propósito de que sean sustituidas posteriormente por las texturas generadas a partir de mallas poligonales. La Figura 2 muestra el flujo de datos entre los distintos pases coordinados por la clase `renderer::PathTracerPipeline` y la Tabla 4 detalla los *Framebuffer Objects* (FBO) y texturas asociados.

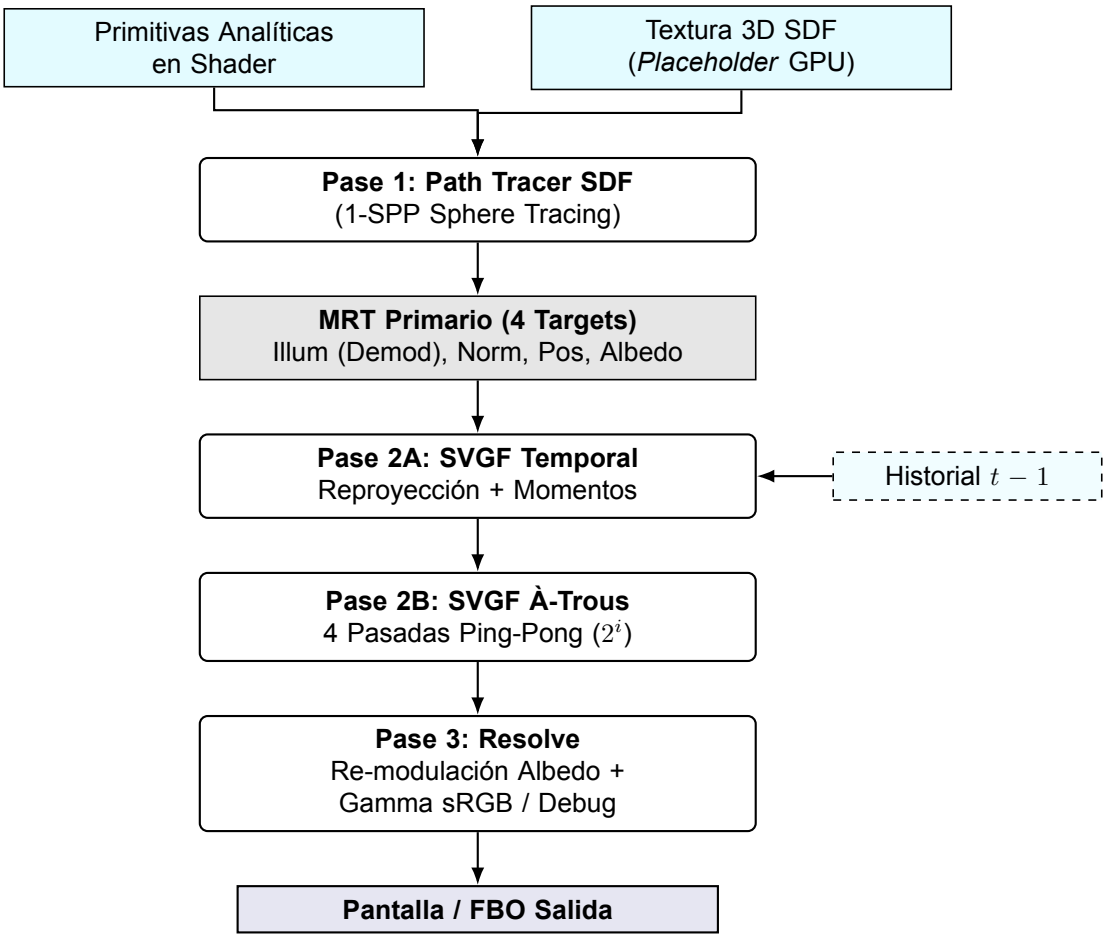


Figura 2: Arquitectura del banco de pruebas visual con evaluación de SDF y reducción de ruido SVGF.

El funcionamiento de esta primera fase del entorno de pruebas comprende los siguientes mecanismos clave:

- **Evaluación con primitivas analíticas y muestreo con ruido azul (STBN):** En el primer paso de implementación del visor (`sdf_raymarching.frag`), las escenas de prueba (como la caja de Cornell y el muestrario de materiales) se definen inyectando primitivas SDF analíticas (esferas, cajas, cilindros y planos combinados mediante operaciones booleanas) directamente en la función de distancias del shader (`fScene`). A partir de la intersección primaria obtenida por *sphere tracing*, se evalúa la iluminación directa con sombras suaves cónicas y un rebote indirecto (difuso por muestreo coseno o especular gobernado por Fresnel-Schlick [13]). Para evitar la aglomeración de ruido en bajas frecuencias propia de los generadores pseudo-

Tabla 4: Configuración de recursos volumétricos y Framebuffer Objects (FBO) en el visor de validación.

Recurso / FBO	Attachment	Textura	Formato interno	Datos almacenados
`m_sdfTexture`	Imagen / Sampler	`Texture3D`	`GL_R32F`	Campo de distancias 3D generado en *compute shader* (*placeholder*).
`m_primaryFbo`	Color 0–3	`m_primary*`	`GL_RGBA16F`	Irradiancia demodulada, profundidad lineal, normal, impacto y albedo.
`m_temporalFbo[2]`	Color 0–1	`m_temporal*`	`GL_RGBA16F`	Irradiancia integrada, varianza temporal, momentos (μ_1, μ_2) y longitud de historial.
`m_atrousPing/Pong`	Color 0	`m_atrous*`	`GL_RGBA16F`	Irradiancia y varianza filtradas mediante filtro *wavelet* À-Trous (5×5).

aleatorios de ruido blanco (PCG), se integran máscaras de ruido azul espaciotemporal (*Spatiotemporal Blue Noise*, STBN) [14, 15] almacenadas en texturas 3D de $128 \times 128 \times 64$ texeles, que distribuyen el error estocástico en frecuencias altas fácilmente suprimibles por el filtro posterior.

- **Visualización de texturas de distancia 3D a modo de *placeholder* (avance del Hito OE2):** Una vez validado el trazador con ecuaciones directas en el shader, se incorpora el soporte para almacenar y muestrear campos de distancia discretos en texturas tridimensionales mediante las clases `Texture3D` y `ComputeShader` junto con el shader `generate_3d_sdf.comp`. En esta fase, dicho *compute shader* rellena texturas `GL_TEXTURE_3D` en formato `GL_R32F` (a resoluciones configurables de 32^3 a 256^3) evaluando también primitivas analíticas codificadas directamente en el propio shader (una esfera de 1 m de diámetro, un conjunto de primitivas y una habitación hueca con ventana mediante operaciones CSG). Estas texturas actúan como *placeholder* funcional para dejar resueltos y verificados el muestreo trilineal continuo en hardware, el cálculo de normales por diferencias centrales adaptadas al tamaño de vóxel (`get3DTextureNormal`) y el *surface bias* proporcional a la resolución (`getSurfaceBias`), de modo que en la siguiente fase puedan sustituirse de forma transparente por las texturas generadas a partir de mallas poligonales.

- **Reconstrucción espacio-temporal SVGF estabilizada:**

  El pase temporal (`svgf_temporal.frag`) reproyecta las muestras mediante la matriz del fotograma anterior validando la coherencia de normales, distancia planar y continuidad de superficie, y actualiza la media exponencial junto con los momentos de luminancia (μ_1, μ_2). En regiones recién desocluidas con historial corto, complementa la estimación de varianza con un vecindario espacial bilateral de 7×7. A con-

tinuación, el filtro espacial (`svgf_atrous.frag`) aplica cuatro iteraciones *ping-pong* con núcleo B_3-spline de 5×5 y pasos $s \in \{1, 2, 4, 8\}$ [12], ponderadas por funciones de preservación de bordes (*edge-stopping functions*) de normal, profundidad planar y luminancia, antes de remodular el albedo y aplicar la corrección gamma en `linear_to_srgb.frag`.

- **Interfaz diagnóstica y verificación del visor:** La interfaz en Dear ImGui permite alternar escenas, configurar luces, cámara y escala de resolución, conmutar entre la evaluación analítica directa y la lectura de la textura 3D *placeholder* a distintas resoluciones, e inspeccionar seis vistas diagnósticas (imagen final reconstruida, entrada bruta a 1 SPP, irradiancia filtrada (*denoised*), varianza estimada, normales y albedo). Sobre una GPU AMD Radeon RX 9060 XT a resolución de 960×540, el filtro SVGF completo consume aproximadamente **1,85 ms** por fotograma, manteniendo tasas superiores a **120 FPS**, y supera la batería automatizada `--test` con **0 píxeles anómalos (`NaN`/`Inf`)**. Tal como muestran la Figura 3 y la Figura 4, el visor reproduce con fidelidad tanto la transferencia indirecta de color (*color bleeding*) en la caja de Cornell como las sombras suaves y reflexiones sobre geometría SDF.

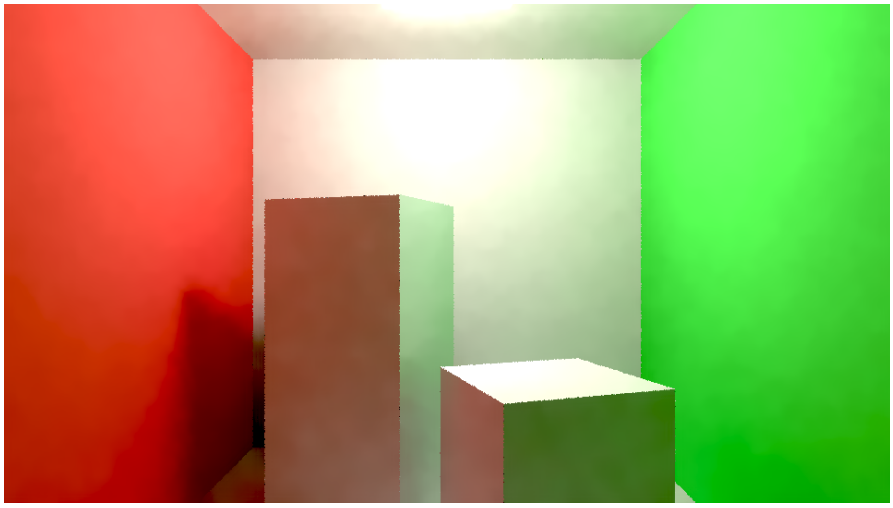


Figura 3: Escena de validación Cornell Box renderizada en tiempo real en el visor a 1 muestra por píxel con reconstrucción SVGF.

3.4. Escenas de evaluación y protocolo experimental

Para la evaluación comparativa de los dos métodos de generación de SDF a partir de mallas poligonales (que se ejecutará en las fases correspondientes a las entregas PEC 3 y PEC 4), se definen los siguientes modelos de entrada y métricas de análisis:

3.4.1. Modelos y escenas poligonales de prueba

1. **Modelos geométricos de control (Esfera y Cubo poligonales):** Mallas sintéticas cerradas teseladas con densidad poligonal variable ($1{,}000$ a $20{,}000$ triángulos) cuyas dimensiones coinciden con funciones SDF analíticas conocidas en forma cerrada:

$$f_{\text{esfera}}(p) = \|p\| - r, \quad f_{\text{cubo}}(p) = \|\,\text{máx}(|p| - b, 0)\| + \text{mín}(\text{máx}(|p| - b), 0) \qquad (15)$$

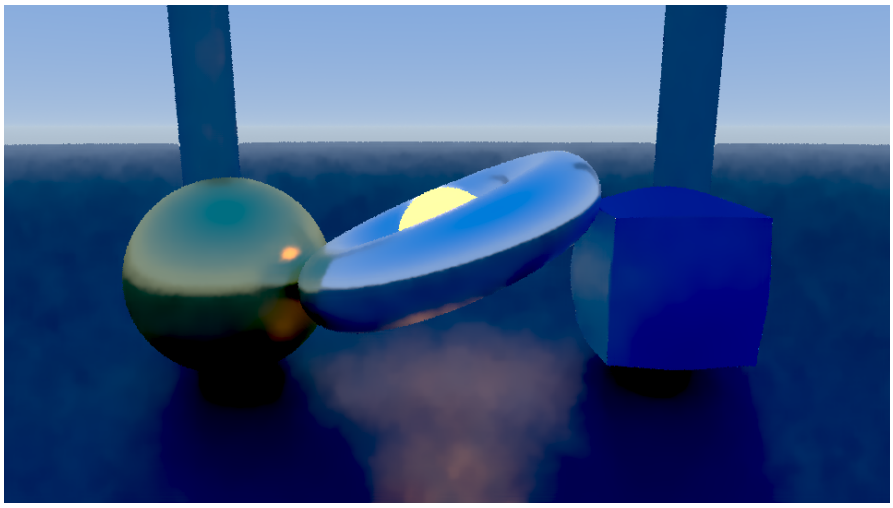


Figura 4: Escena de validación con muestrario de primitivas SDF, reflexiones especulares y sombras suaves.

Estas geometrías permitirán comprobar numéricamente la exactitud del generador base en *compute shaders* antes de emplear sus volúmenes como referencia.

2. **Cornell Box poligonal:** Versión discretizada en triángulos de la caja de Cornell tradicional, utilizada para evaluar el comportamiento de ambos generadores en recintos cerrados con ángulos diedros rectos y oclusiones controladas.
3. **Crytek Sponza:** Escena arquitectónica compuesta por aproximadamente $262{,}000$ triángulos con arcos, columnas y detalles geométricos complejos, destinada a evaluar la escalabilidad temporal y el consumo de memoria ante un número elevado de primitivas.

3.4.2. Protocolo de medición y métricas cuantitativas

Los dos generadores se compararán bajo condiciones idénticas de GPU, formato de almacenamiento (`GL_R16F` y `GL_R32F`) y resolución volumétrica (64^3, 128^3 y 256^3 vóxeles), registrando tres métricas principales:

1. **Tiempo de generación en GPU (ms):** Se aislará el tiempo de ejecución de los *compute shaders* de generación respecto a la CPU y al renderizado del visor mediante objetos de consulta asíncrona de OpenGL (`GL_TIME_ELAPSED`).
2. **Consumo de memoria de vídeo VRAM (MB):** Se contabilizará la memoria estática ocupada por la textura volumétrica de salida (`GL_TEXTURE_3D`) junto con la memoria temporal adicional requerida durante la generación (*buffers* SSBO de geometría y texturas *ping-pong* de coordenadas de semillas en JFA 3D).
3. **Error numérico del campo de distancias:** Tomando el volumen generado por el método base de cálculo exacto punto-triángulo como campo de referencia $f_{\text{ref}}(i, j, k)$, se evaluarán vóxel a vóxel sobre los V^3 elementos de la cuadrícula las siguientes métricas de discrepancia frente al volumen obtenido mediante JFA $f_{\text{JFA}}(i, j, k)$:

- **Error Absoluto Medio (MAE, *Mean Absolute Error*):**

$$\text{MAE} = \frac{1}{V^3} \sum_{i=1}^{V_x} \sum_{j=1}^{V_y} \sum_{k=1}^{V_z} |f_{\text{JFA}}(i,j,k) - f_{\text{ref}}(i,j,k)| \tag{16}$$

- **Error Cuadrático Medio (RMSE, *Root Mean Square Error*):**

$$\text{RMSE} = \sqrt{\frac{1}{V^3} \sum_{i=1}^{V_x} \sum_{j=1}^{V_y} \sum_{k=1}^{V_z} (f_{\text{JFA}}(i,j,k) - f_{\text{ref}}(i,j,k))^2} \tag{17}$$

- **Error Máximo Absoluto ($E_{\text{máx}}$):**

$$E_{\text{máx}} = \max_{i,j,k} |f_{\text{JFA}}(i,j,k) - f_{\text{ref}}(i,j,k)| \tag{18}$$

4. Diseño

(Este capítulo se desarrollará en la PEC 3)

5. Resultados

(Este capítulo se desarrollará en la PEC 3 y PEC 4)

6. Conclusiones y Trabajo futuro

6.1. Conclusiones

(Este capítulo se desarrollará en la PEC 4)

6.2. Seguimiento de la planificación

(Este apartado se desarrollará en la PEC 4)

6.3. Líneas de futuro

(Este apartado se desarrollará en la PEC 4)

7. Glosario

BRDF (*Bidirectional Reflectance Distribution Function*): Función matemática de cuatro dimensiones que define cómo la luz se refleja en una superficie opaca en función de las direcciones incidente y saliente.

Compute Shader: Etapa de *shader* programable en la GPU no ligada al *pipeline* gráfico tradicional de rasterización, que permite ejecutar cálculos masivamente paralelos de propósito general (GPGPU) sobre *buffers* y texturas arbitrarias.

DSRM (*Design Science Research Methodology*): Metodología formal de investigación orientada a la creación, desarrollo y evaluación iterativa de artefactos tecnológicos prácticos.

FBO (*Framebuffer Object*): Objeto de OpenGL que agrupa destinos de renderizado fuera de pantalla (texturas de color, profundidad y stencil) para procesamiento diferido.

G-Buffer (*Geometry Buffer*): Conjunto de texturas en espacio de pantalla que almacenan información geométrica (posiciones, normales, albedo, rugosidad) generada por el pase de geometría en renderizado diferido.

JFA (*Jump Flooding Algorithm*): Algoritmo paralelo de propagación que aproxima transformadas de distancia euclídea y diagramas de Voronoi en cuadrículas discretas en un número logarítmico de pasos iterativos de salto ($\mathcal{O}(V^3 \log_2 V)$).

MRT (*Multiple Render Targets*): Característica de las GPUs y APIs gráficas que permite a un shader de fragmentos escribir simultáneamente en múltiples texturas en una única pasada de renderizado.

Path Tracing: Algoritmo de renderizado basado en integración de Monte Carlo que simula numéricamente el transporte global de la luz mediante el trazado de trayectorias estocásticas de rayos.

Ray Marching: Técnica de trazado de rayos iterativa en la que el rayo avanza en incrementos discretos a lo largo de su vector director hasta intersectar un umbral de proximidad a la superficie.

SDF (*Signed Distance Field*): Campo escalar continuo o discretizado que almacena la distancia euclídea mínima desde cualquier punto del espacio a la superficie geométrica más cercana, codificando el interior mediante valores negativos y satisfaciendo la ecuación *eikonal*.

Sphere Tracing: Variante acelerada de *ray marching* aplicable a funciones SDF en la que la magnitud del paso de avance es exactamente igual a la distancia devuelta por el campo en la posición actual del rayo.

SPP (*Samples Per Pixel*): Número de trayectorias de rayos primarios muestreadas estocásticamente por cada píxel de la imagen en un fotograma determinado.

SSBO (*Shader Storage Buffer Object*): *Buffer* de memoria en GPU accesible en lectura y escritura por los shaders, caracterizado por su gran capacidad de almacena-

miento y soporte para estructuras complejas de datos con alineamiento estandarizado (`std430`).

SVGF (*Spatiotemporal Variance-Guided Filtering*): Técnica de reconstrucción de imágenes en tiempo real para iluminación global que combina acumulación temporal, recorte de historial y filtrado espacial *wavelet* À-Trous ponderado por varianza.

Voxelización: Proceso de discretización espacial que convierte una representación de superficie geométrica continua (como una malla de triángulos) en una aproximación volumétrica compuesta por elementos cúbicos de volumen (*vóxeles*) sobre una cuadrícula regular tridimensional.

VRAM (*Video Random Access Memory*): Memoria física de alta velocidad integrada en la tarjeta gráfica dedicada a almacenar texturas, *buffers* de geometría y destinos de renderizado.

8. Bibliografía

[1] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann Publishers, San Francisco, CA, Estados Unidos, 2004.

[2] M. Macklin, K. Erleben, M. Müller, N. Chentanez, S. Jeschke y Z. Corse, «Local optimization for robust signed distance field collision», *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 3, n.º 1, artículo 8, pp. 1–17, 2020. doi: 10.1145/3384538.

[3] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart y J. Nieto, «Voxblox: Incremental 3D Euclidean Signed Distance Fields for on-board MAV planning», en *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, BC, Canadá, 2017, pp. 1366–1373. doi: 10.1109/IROS.2017.8202315.

[4] J. C. Hart, «Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces», *The Visual Computer*, vol. 12, n.º 10, pp. 527–545, 1996. doi: 10.1007/s003710050084.

[5] J. T. Kajiya, «The rendering equation», en *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*, Association for Computing Machinery, Nueva York, Estados Unidos, 1986, pp. 143–150. doi: 10.1145/15922.15902.

[6] D. Wright, K. Narkowicz y P. Kelly, «Lumen: Real-Time Global Illumination in Unreal Engine 5», en *ACM SIGGRAPH 2022 Courses: Advances in Real-Time Rendering in Games*, 2022. Disponible en: https://advances.realtimerendering.com/s2022/index.html [Consulta: 24 de octubre de 2026].

[7] G. Rong y T.-S. Tan, «Jump flooding in GPU with applications to Voronoi diagram and distance transform», en *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games (I3D '06)*, Association for Computing Machinery, Nueva York, Estados Unidos, 2006, pp. 109–116. doi: 10.1145/1111411.1111431.

[8] K. Peffers, T. Tuunanen, M. A. Rothenberger y S. Chatterjee, «A Design Science Research Methodology for Information Systems Research», *Journal of Management Information Systems*, vol. 24, n.º 3, pp. 45–77, 2007. doi: 10.2753/MIS0742-1222240302.

[9] J. A. Bærentzen y H. Aanæs, «Signed distance computation using the angle weighted pseudonormal», *IEEE Transactions on Visualization and Computer Graphics*, vol. 11, n.º 3, pp. 243–253, 2005. doi: 10.1109/TVCG.2005.49.

[10] A. Jacobson, L. Kavan y O. Sorkine-Hornung, «Robust inside-outside segmentation using generalized winding numbers», *ACM Transactions on Graphics (SIGGRAPH 2013)*, vol. 32, n.º 4, artículo 33, pp. 1–12, 2013. doi: 10.1145/2461912.2461916.

[11] C. Schied, A. Kaplanyan, C. Wyman, A. Patney, C. R. A. Chaitanya, J. Burgess, S. Liu, C. Dachsbacher, A. Lefohn y M. Salvi, «Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination», en *Proceedings of High Performance Graphics (HPG '17)*, Association for Computing Machinery, Nueva York, Estados Unidos, 2017, artículo 2, pp. 1–12. doi: 10.1145/3105762.3105770.

[12] H. Dammertz, D. Sewtz, M. Hanika y H. P. A. Lensch, «Edge-avoiding À-Trous filtering for fast global illumination and image reconstruction», en *ACM SIGGRAPH 2010 Talks*, Association for Computing Machinery, Nueva York, Estados Unidos, 2010. doi: 10.1145/1837026.1837050.

[13] B. Karis, «Real Shading in Unreal Engine 4», en *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice*, 2013. Disponible en: https://blog.selfshadow.com/publications/s2013-shading-course/karis/s2013_pbs_epic_notes_v2.pdf [Consulta: 24 de octubre de 2026].

[14] A. Wolfe, N. Morrical, T. Akenine-Möller y R. Ramamoorthi, «Spatiotemporal Blue Noise Masks», en *Eurographics Symposium on Rendering (EGSR) - Research Track*, Eurographics Association, 2022. doi: 10.2312/sr.20221161.

[15] A. Wolfe, «Rendering in Real Time with Spatiotemporal Blue Noise Textures, Part 1», *NVIDIA Technical Blog*, 2022. Disponible en: https://developer.nvidia.com/blog/rendering-in-real-time-with-spatiotemporal-blue-noise-textures-part-1/ [Consulta: 24 de octubre de 2026].

9. Anexos

(No se incluyen anexos en esta entrega)